WEEK 2

Distributed Systems Principles for Data

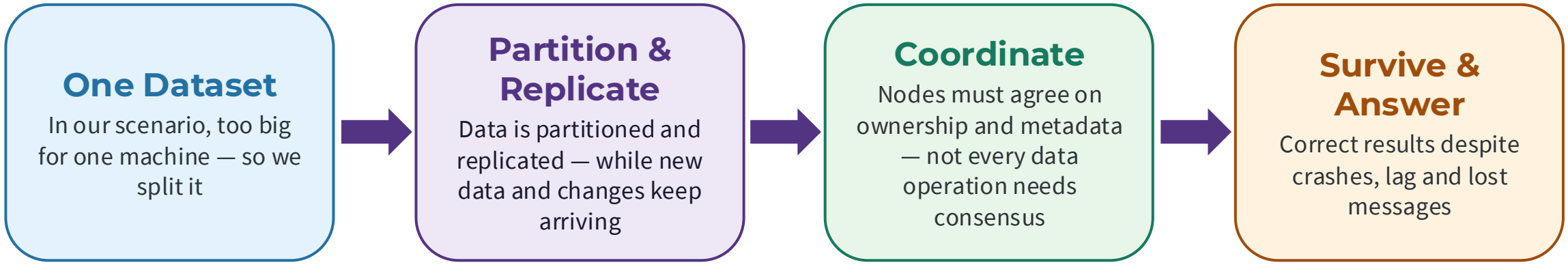
Partitioning, Replication, Consistency & the Art of Trade-offs

Partition & Replicate
How data spreads safely over many machines

Consistency & Consensus
What correct means when nodes disagree

CAP & PACELC
The trade-offs every system must choose

Why Distribution Needs Principles



1. Split the data

How do we split data no single machine can hold?

2. Copy it — safely

How do replicas remain consistent as the data evolves?

3. Agree on state

How do nodes agree on order, ownership and truth?

4. Tolerate failures

How do we stay correct despite crashes and lost messages?

Every distributed data stack has to answer these four questions

Spark, Kafka and the lakehouse each address them somewhere — this week we learn the questions.

Contents

Week 2 — how data is split, copied and kept consistent across machines, and the trade-offs behind it

01 Partitioning & Replication

Partitioning & sharding strategies
Rebalancing, routing & data skew
Replication topologies
Quorums — W + R > N

02 Consistency & Consensus

The consistency spectrum
What clients actually notice
Why agreement is hard
Raft in outline

03 Failures & Trade-offs

Failure models & fault tolerance
Designing for failure
CAP & PACELC — Consistency, Availability, Partition tolerance
Where real systems sit

04 Lab — Week 2

Partitioning & skew on containers
Queries on 1–8 nodes, failures & quorums
AI session: benchmark its schemes
Next week: storage for scale

Scenario — One Table, 100 Machines

Split a 2 TB user-events table across a cluster — every principle of this week appears at once

The Question

"How do we split one table over 100 machines — and still trust the answers?"

- Split it wrong → one node does all the work while 99 sit idle
- Copy each piece **3×** for safety — now which copy is the truth?
- A node dies mid-query — who notices, and who takes over?
- And two clients may read different values at the same instant

The Napkin Sketch

```
# 2 TB table, 100 nodes:
#   ≈ 20 GB primary data per node, if even
#   3 replicas → 6 TB, ≈ 60 GB per node + overhead
# but reality asks more:
four questions, every single time:
how to split?  — partitioning
  how to copy?  — replication
  which copy is true?  — consistency
  who decides? — consensus
→ this week: the four answers
```

The lesson

Distribution is a set of choices, not an accident — every system you will meet made these four choices differently. This week you learn to read them.

Partitioning — Splitting the Data

1 — By Range
ordered keys: A–F, G–N, O–Z

2 — By Hash
hash(key) mod N — N partitions, not nodes

3 — By Directory
a lookup service maps key → shard

Three strategies — and the key that decides

```
// range — keeps order, risks hot spots
shard_1 = keys [A..F]  →  range scans are cheap
but today's keys cluster together
  timestamp keys ⇒ all writes hit the last shard
  → the classic hot-spot trap

// hash — spreads keys, not hot keys; loses order
shard = hash(user_id) mod 100 // 100 partitions
distinct keys spread evenly across shards
  but: range queries now touch every shard
  → scatter-gather reads

// directory — a lookup table decides; flexible, but one more hop
shard = lookup[key]        // service maps key → shard
move a hot key by editing one row — no rehash
  but: the lookup is on every path
  → cache it, replicate it
```

Choosing a partition key

- ✓ The key decides where every record lives
- ✓ Aim for even load and even data — a frequent key still lands on one shard
- ✓ Match the key to the dominant query pattern
- ✓ Composite keys (hash, then range) keep order within each hash group
- ✓ Changing a poor partitioning scheme later can be expensive — choose early, choose well

Partition = how you split the data; **shard** = where a partition lives.

Kafka topics (streaming log), Cassandra rings (wide-column store), Spark shuffles (batch engine) and lakehouse tables (Parquet/Delta on object storage) all begin with the same decision: what is the partition key?

The Ring — Nodes and Keys

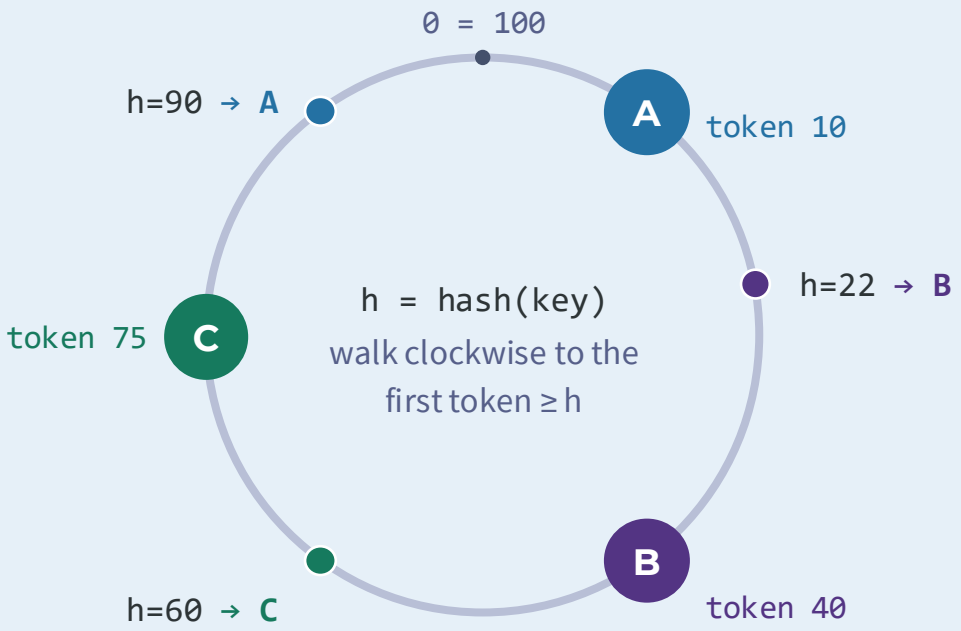
PROBLEM `hash(key) mod N`: change N, most keys move
SOLUTION hash node IDs onto the same circle

1 · BUILD THE RING

```
ring = []
for n in nodes:                          # A, B, C
    ring.append((hash(n) mod 100, n))    # 10,40,75
ring.sort()
# the ring IS this sorted list
```

2 · FIND THE OWNER OF A KEY

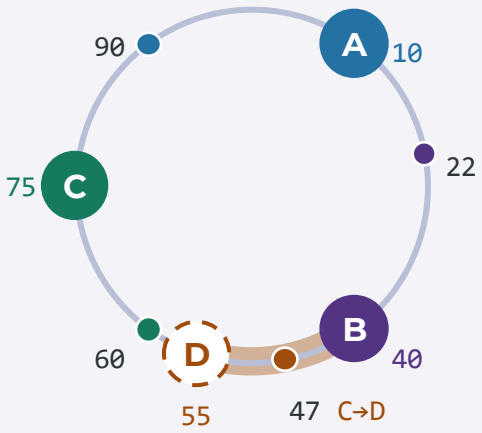
```
def owner(key):
    h = hash(key) mod 100
    return node of first token ≥ h
    # none ≥ h?  wrap: first token in the ring
```



Same hash function for keys and node IDs, so both land on one circle. Values 0–100 stand in for the real 2^{64} hash space.

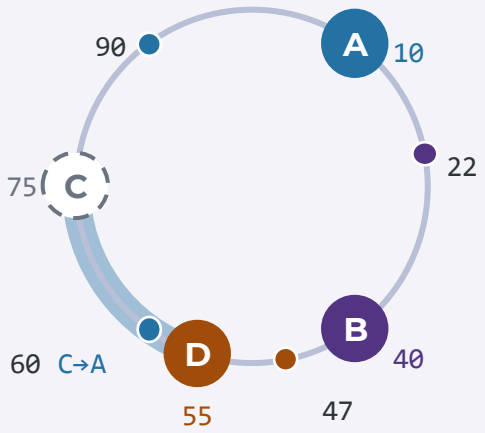
The Ring — Nodes Join and Leave

D joins at 55



only (40, 55] changes owner

C leaves



(55, 75] goes to successor A

D joins at token 55

`[(10, A), (40, B), (55, D), (75, C)]`

- D takes the arc (40, 55] from C: key **47** moves **C → D**
- Keys 22, 60, 90 keep their owner: 1 of 4 moved ≈ 1/(N+1)
- **`mod N`** going 3 → 4 would move 3 of 4

C leaves

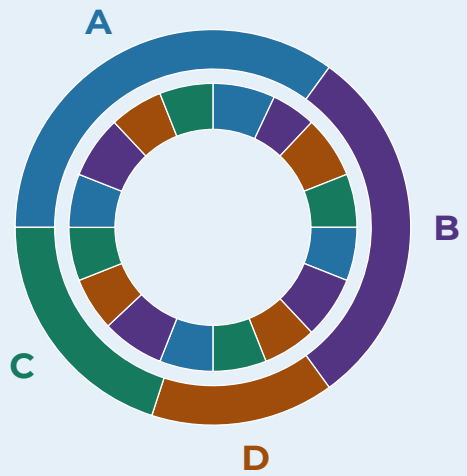
`[(10, A), (40, B), (55, D)]`

- C's arc (55, 75] goes to its successor: wrap → **A**; key **60** moves **C → A**
- The ring bounds how much moves, not how: streaming data and updating clients is a migration & routing protocol

Inserting or deleting one (token, node) tuple in the sorted list only touches the arc next to it — every other key's owner is unchanged.

Rebalancing & Virtual Nodes

One token per node → uneven arcs



4 tokens per node

`hash("X-i") mod 100`

```
A-0..3 →  7 31 56 81
B-0..3 → 12 38 63 88
C-0..3 → 25 50 75  0
D-0..3 → 19 44 69 94
```

Outer, 1 token: 35 · 30 · 15 · 20
Inner, 4 tokens: 25 · 26 · 24 · 25

- **Virtual nodes:** more tokens → arcs average out; a bigger machine takes more tokens
- They cannot split one hot key; rebalancing while serving still needs a migration & routing protocol

The Vocabulary

- **Shard** → a logical partition; its replicas may live on several nodes
- **Rebalancing** → shards move as nodes come & go
- **Consistent hashing** → a ring limits movement
- **Virtual nodes** → many tokens per node even out the arcs
- **Routing tier** → requests find the shard
- **Hot spot** → one shard, outsized load
- **Skew** → uneven data or load

Cassandra: 256 virtual tokens per node. Kafka assigns topic partitions explicitly (not a ring); HBase splits regions by key range.

Data Skew — the Hidden Enemy

Perfect partitioning on paper still fails in production — real keys are often skewed

Uniform — the textbook picture

```
# 100 shards, even work, 1 stage
each shard ≈ 20 GB
  p99 close to p50 shard time
  stage time ≈ slowest ≈ mean
  near-linear if work is uniform
```

Skewed — the real picture

```
# real keys, ≈ equal bytes/event
top user = 4% of all events
  one shard: ≈100 GB — hot
  99 shards under-used, one melts
  stage time = the straggler
```

Where skew comes from

```
zipf      = few keys dominate
time      = today's shard is hot
celebrity = one key > one shard
joins     = shuffles amplify
```

✓ **One hot key → salt it**

✓ **One hot range → split it**

✓ **Monitor per-shard load**

*Skew is uneven data or work across partitions — measure rows, bytes and task time. **Salting** appends a random suffix to one heavy key so it lands on several shards (queries merge by the original key); a range that is heavy because of many keys is simply divided in two. **monitoring** finds skew before your users do. Lab 2 measures both.*

Replication — Copies for Safety

1 — Leader–Follower
writes to one node; reads can scale out

2 — Multi-Leader
write in many places; resolve conflicts

3 — Leaderless
quorum reads & writes — Dynamo-style

Why keep three copies of everything?

```
// availability — survive node loss
replicas = 3  →  survives 2 lost copies, once on all 3
  but a majority quorum needs 2 live → 1 down
reads scale out across copies that may lag
  ack policy: which replicas confirm — none / some / quorum?
  → un-acked followers apply later — lag

// the cost — copies drift
leader commits; followers apply later
a follower lags — its reads go stale
  the leader crashes → failover
  → who is the new leader? (consensus)
```

Choosing a topology

- ✓ Leader–follower: simple write ordering — one write point
- ✓ Multi-leader: multi-region writes — conflicts to resolve
- ✓ Leaderless: no leader election — needs enough reachable replicas
- ✓ Sync replicas confirm before the ack; freshness also depends on the read protocol
- ✓ Every choice reappears in Kafka, Cassandra & the lakehouse

Replication is never free: you buy availability and pay in lag, conflicts or coordination — the next slides put a price on each.

Quorums — Overlapping Replica Sets

Leaderless replication avoids leader election, but needs enough reachable replicas — and W + R > N

1. Choose N
N replicas hold each record (the item's replica set)

2. Write W
A write succeeds once W replicas confirm

3. Read R
A read needs R responses, then reconciles the versions

4. Overlap
W + R > N ⇒ every read set overlaps every write set

quorum arithmetic — the standard setup

```
N=3 · W=2, R=2  →  2+2 > 3 ✓
  every read set overlaps every write set
  tolerates: 1 slow or dead replica
N=3 · W=1, R=1 → fast, but:
  1+1 < 3 — stale reads possible
N=3 · W=3, R=1 → 3+1 > 3 ✓ but:
  W=N: one dead replica ⇒ no writes
tune W, R: latency / staleness / availability
⚠ Overlap alone does not guarantee linearizability
// versions must reconcile · Cassandra: CL/query
```

What quorums do not fix

- **Sloppy quorums** — under stress, writes land on stand-in nodes
- **Concurrent writes** — quorums order nothing; conflicts remain
- **Repair lag** — replicas may stay stale until read repair, hints or scheduled repair
- **No transactions** — each key is protected by its own quorum; debit A and credit B, and a reader can see the money leave A before it reaches B

The Consistency Spectrum

What may a client see while replicas converge? Four common contracts, from strict to loose

1

Linearizable

The one-copy illusion: each operation takes effect atomically between its invocation and response, in real-time order. An overlapping write may land before or after a read.

2

Sequential

All clients observe one sequential history that preserves each client's program order — real-time order across clients need not hold.

3

Causal

Effects never precede causes: replies always follow the messages they answer. Preserves dependencies and can reduce coordination.

4

Eventual

If updates stop and communication resumes, replicas eventually converge — given propagation and reconciliation. No promise about the meantime.

Why this matters

- Stronger guarantees can require coordination or timing assumptions; their latency cost depends on protocol, topology and workload. Weaker models push conflict handling into your application code.
- **Choose per operation** where supported, not per system — a shopping basket and a bank transfer deserve different contracts.

What Clients Actually Notice

```
# replication lag, seen from a browser
you post a comment — it hits the leader
you refresh — the read hits a follower
    the follower lags 2 s behind
    your comment has vanished(?)
# one fix: read-your-writes
    route to a replica that has your write

# other anomalies feel like time travel:
# comments reorder; answers appear before
# questions — track versions
```

Session Guarantees

- **Read-your-writes** → you see your own updates
- **Monotonic reads** → reads never regress
- **Monotonic writes** → your writes apply in order
- **Writes-follow-reads** → writes follow reads

Other client-visible guarantees

- **Consistent prefix** → a prefix of a stated history
- **Bounded staleness** → lag ≤ Δ time or versions

Implementation technique

- **Affinity + versions** → replica has your state

Session guarantees are promises to one client, not a global order — cheaper than linearizability, often what the product needs; cross-client causality needs more.
Writes-follow-reads: a write you make after reading something is ordered after what you read, on every replica

Consensus — Split Brain

Leader-based replication needs agreement on who leads — the hardest problem in distributed systems

The Situation

"The leader stopped answering. Is it dead — or just slow?"

- In an asynchronous network you cannot tell a crash from lag
- Elect a new leader **`too early`** — and the old one may still be alive
- Two leaders independently acknowledging conflicting writes = split brain — committed histories diverge
- Wait too long, and the system is simply down

The Failure, Sketched — naïve failover

```
# t0: the network partitions
#   {A,B}  |  {C,D,E}   leader = A
#   C,D,E elect C — no quorum check, no fencing
# both keep accepting writes:
A ← balance = 70
C ← balance = 20
  partition heals → conflict
  which history wins?
  ad-hoc fixes fail in subtle ways
→ we need a protocol with proofs
```

The lesson

Majorities are the ingredient: only a group holding more than half the nodes may decide, and two disjoint majorities cannot exist. Consensus adds the rules that make that safe — elections with terms, voting restrictions, commit rules — and Raft packages them.

Raft — Consensus in Outline

One protocol, four moves — designed to be understandable, and now everywhere in infrastructure

1

Heartbeat timeout

Followers expect regular heartbeats from the leader. Silence past a randomised timeout starts an election.

2

Leader election

A candidate becomes leader on a majority of votes — at most one leader per term. A split vote can end the term with no leader; candidates retry in a later term.

3

Log replication

All writes flow through the leader, which appends them to its log and replicates them to followers, in order.

4

Commit on majority

An entry from the leader's current term, once durably stored on a majority, is committed — it survives any minority of crash failures. Older-term entries commit only indirectly (Raft §5.4.2).

Why this matters

- etcd (under Kubernetes), Kafka's KRaft metadata quorum, CockroachDB and cloud control planes all run on Raft or a sibling (Paxos, ZAB).
- This week's primary reading: “In Search of an Understandable Consensus Algorithm” — Raft, from its authors.

Failures — Know Your Enemy

FAULT MODELS · what goes wrong

CRASH

Crash-stop

A node halts and stays halted (fail-stop if reliably detected); crash-recovery lets it restart from persistent state

OMISSION

Lost messages

Requests or replies silently vanish — networks drop, buffers overflow

TIMING

Slow ≠ dead

GC pauses and overload mimic crashes — timeouts are guesses, not proof

PARTITION

Split network

Sides keep running, unaware of each other — may be asymmetric or partial; the CAP scenario made real

FAULT MODELS (cont.)

BYZANTINE

Arbitrary faults

Nodes lie or corrupt data — tolerating them needs a protocol designed for that model (e.g., PBFT); crash-fault Raft does not cover it

RESPONSES · what you do about it — detail on slide 17

DETECTION

Heartbeats

Periodic pings plus timeouts yield suspicion — never certainty

REDUNDANCY

No single point

Replicas, retries and idempotent operations absorb individual faults

RECOVERY

Rejoin & repair

Restore a compatible checkpoint, replay the needed log; catch up, then serve again

Everything fails, all the time — the craft is designing systems where partial failure is normal operation, not an emergency.

Designing for Failure

DETECT & SUSPECT

Timeouts & heartbeats

you cannot know a node is dead — only decide to stop waiting

- **Timeouts everywhere** every remote call gets a budget — waiting forever is not an option
- **Heartbeats & gossip** nodes advertise liveness; peers spread suspicion gradually
- **Fencing tokens** the protected resource rejects writes carrying a stale epoch number

TOLERATE & RECOVER

Redundancy & retries

failures keep coming — the system must absorb them and keep serving

- **Retries + idempotency** retry within a deadline, with backoff, jitter and a retry budget; make effects idempotent or atomically dedupe by request ID
- **Replicas & failover** a standby takes over; quorum election plus fencing keeps two leaders from both committing
- **Checkpoints & logs** recovery restores a compatible checkpoint, then replays the required log

Golden rule: ***partial failure is the normal case — design for it.***

Spark retries tasks and restages work; Kafka re-elects partition leaders — in the coming weeks you will watch these mechanisms fire.

CAP — Pick Two? Not Quite

During a partition, a replicated service cannot guarantee both linearizability and completion of every request at every non-failing node

CAP — the three letters

```
C  = Consistency (linearizable)
A  = Availability (must answer)
P  = Partition tolerance

CA = assumes no partitions
CP, AP = the real choices under P
choice = partition-time only
```

CP — reject or delay, stay correct

```
# partition strikes:
no quorum? reject/delay R & W
  errors beat unverified data
  ledgers, locks, configs
  e.g. HBase, etcd def. reads
```

AP — answer, weaker C

```
# partition strikes:
reachable replicas serve
  reads may be stale (contract)
  carts, feeds, telemetry
  e.g. Dynamo, Cassandra @ONE
```

✓ **P cannot be prevented**

✓ **Classify per operation**

✓ **Theorem exact, practice varies**

*CAP names the partition case only. **CP** (C & P) may reject or delay operations it cannot verify; **AP** (A & P) accepts weaker consistency so operations continue. Outside partitions, latency vs consistency — PACELC, next slide.*

PACELC — the Full Price List

Partition?
The network splits — or only looks split; a node cannot tell instantly

The branch, per operation

Yes → A or C
Keep answering under weaker consistency — or reject/delay to stay consistent

Availability vs consistency — the CAP case (slide 16)

No (Else) → L or C
Answer fast from the nearest replica — or coordinate for consistency

Latency vs consistency — the everyday replication trade-off

Outside partitions, replication still trades latency against consistency
Labels describe particular configurations: Abadi (2012) discusses Dynamo as PA/EL and HBase as PC/EC. Evaluate modern MongoDB by its read concern, write concern, read preference and replica-set topology.

Where Real Systems Sit

Five examples, one vocabulary — labels describe configurations, not products

Cassandra
hash ring · leaderless · PA/EL (at low CL)

DynamoDB
leader per partition · eventual or strong reads where supported

HBase
range shards · one serving server per region · PC/EC

etcd / ZooKeeper
Raft & ZAB · CP · read freshness varies

Kafka
partitioned log · ISR replication · KRaft metadata quorum

The four questions, answered



Partition
Hash rings (Cassandra) vs ordered ranges (HBase) vs fixed topic partitions (Kafka)



Replicate
Leaderless quorums vs leader–follower replication vs Kafka's in-sync replica sets



Promise
Tunable per query in Cassandra; linearizable by default in etcd (serializable reads optional) — consistency is a dial



Agree
Raft inside etcd & Kafka's KRaft metadata quorum; ZAB inside ZooKeeper — quorums everywhere

`Same four questions every time` — *when you meet a new datastore, ask how it partitions, replicates, promises and agrees. The answers are its datasheet.*

Notes: Cassandra PA / EL : under P (partition) stay A (availability); Else favour L (latency) – analogous in Hbase. Cassandra CL (consistency level) : number of writer acknowledge. ZAB - ZooKeeper Atomic Broadcast KRAFT: Kafka Raft).

Reading List

Ranked by priority — the lab assumes the primary set; the secondary set goes deeper on the slides listed.

1 · Primary reading — this week

1 **Kleppmann & Riccomini, *DDIA 2e*, ch. 7**
Sharding, rebalancing & skew, before the lab · Slides 5–9

2 **Kleppmann & Riccomini, *DDIA 2e*, ch. 6**
Replication: leaders, lag & quorums · Slides 10–11, 13

3 **Ongaro & Ousterhout, *Raft, extended version*, 2014**
Figure 2 and §5: elections, log, commit · Slide 15

4 **Kleppmann & Riccomini, *DDIA 2e*, ch. 9–10**
Faults, consistency & consensus, by week 3 · Slides 12, 14–19

2 · Secondary reading — go deeper

1 **DeCandia et al., *Dynamo*, SOSP 2007**
Rings & quorums in production · Slides 6–8, 11

2 **Brewer, *CAP Twelve Years Later*, IEEE Computer 2012**
CAP without the folklore · Slides 18–19

Try this with the AI assistant

Ask it for five exam questions on DDIA ch. 6, answer them, then check its corrections against the book.

Read chapter 7 before the lab if you can: it is the direct companion to Part A. Links: course repo reading list.

PART 2 · HANDS-ON

Lab

02

Lab 2 — TPC-DS on Containers

Before class — at home, ≈30 min

- ✓ **Lab 1 data — SF10 Parquet**
  store_sales and store_returns in $T/parquet/sf10; sf1 also works if SF10 does not fit
- ✓ **Docker with Compose**
  At least 4 CPUs and 4 GB for Docker; on Windows, run every command inside WSL
- ✓ **lab2_starter.zip**
  Build the image; coord status must show w1 to w8 up

compose.yaml — one cluster on your laptop

```
w1 … w8:  DuckDB workers · ≤1 CPU, 1 GB
kv1 … kv3: key-value replicas · W, R
client:    coordinator.py · kv_client.py
lab2_data: partitioned store_sales

dc up -d; coord status # all up
docker pause w3        # a frozen node
docker kill w3         # a crashed node
coord run q3 --nodes 8 # scatter, merge, check
kv bench --w 2 --r 2   # count stale reads
--mbit 100 --delay-ms 5 # simulated network
```

Docker Compose | TPC-DS SF10 | DuckDB 1.4.5 | AI assistant | ai-log.md

Every node is a container on one laptop: shared cores, memory and disk, and a simulated network.

Lab 2 — Tasks & Deliverables

100 minutes in pairs, then take-home work: partition, query on 1 to 8 nodes, replicate, break things

Four parts in class

```
A partition — skew, salting, the ring
B query     — 1→8 nodes, merges, stragglers
C replicate — crash vs freeze, quorums, split
D discuss   — one layout for q1, q2, q3
  /* every result checked: SAME or DIFFERENT */
  /* median of 3 runs wherever time is asked */
→ evidence beats opinion: keep the numbers
```

Deliverables — by the second class after the lab

- Report, PDF, ≤ 4 pages: the tables of Parts A–C, the speedup plot with an Amdahl estimate, your answers
- AI session (take-home): 3 schemes, predictions against measurements, a verdict
- ai-log.md, Session 02 entry: the brief, the answer and what you did with it
- Be ready to explain any number of your report in class

The 100 minutes in class:

A · Partition 25' | **B · Query 30'** | **C · Replicate 25'** | **D · Discuss 10'**

AI Session — Generate & Critique 1/2

session-02.md — the structured brief

```
## role — distributed-data architect, sceptical reviewer
## task — 3 schemes for store_sales, then attack each
## context:
  - TPC-DS SF10, 28.8 M rows · 8 nodes, 1 partition each
  - your Part A table: max/mean per rule + top keys
  - q1 top customers · q2 per store · q3 join
## constraints:
  - one DuckDB SQL expression ≥ 0, mod 8
  - no sampling at query time, no extra tables
## output:
  1. the SQL expression  2. max/mean as a range
  3. the correct plan per query  4. how it fails
# fill every <...> from Part A, then send it
```

What to notice

✓ **The role recruits a critic**
Ask it to attack its own schemes: critique exposes failure modes; the benchmark decides

✓ **Your Part A numbers do the work**
Measured max/mean and hot keys force schemes that fit this data, not textbook data

✓ **Predict before you run**
Write down each max/mean range and each plan before measuring: falsifiable by design

Part 1 of 2, take-home: generate & critique. Next slide: the benchmark that decides which scheme survives contact with the data.

AI Session — The Benchmark 2/2

Worked example — illustrative. Assumed: SF10, 8 nodes, one partition each, median of 3 runs; your numbers will differ

the assistant returned: 3 schemes + predictions
`+ A hash(customer) · B hash(ticket) · C = A, NULL rows salted by ticket`
`+ a plan per query, a failure per scheme · baseline: hash(store)`

✓ Confirmed in this example

- **`C balances the rows`** — max/mean 1.00 against 1.32 for A: the NULL-customer rows spread out
- **`A keeps q1 local`** — pushdown top 10 is SAME: each customer lives on one node
- **`B keeps q3 co-located`** — same rule as store_returns, so no broadcast

✗ Refuted in this example

- **`"store spreads load" (baseline)`** — false: max/mean 1.95, few distinct stores
- **`"pushdown still works on C"`** — DIFFERENT: the NULL customer is split over all nodes
- **`"A can join returns locally"`** — DIFFERENT: returns are split by ticket; A needs a broadcast

The schemes take the assistant a minute; the benchmark is the deliverable. ***`Trust numbers, not confidence.`***

FAQs & Before Next Week

Questions you'll ask anyway

- **Is CAP really "pick two"?** — no; never assume partitions cannot occur; the A-vs-C choice applies only during one, per operation
- **Must I memorise Raft?** — no; explain elections (at most one leader per term), majority commit, and why majorities work
- **Will we implement consensus?** — not from scratch; you will operate it inside Kafka and etcd soon enough
- **Which consistency for the project?** — decide per operation where the system allows, and document the choice in your design
- **Office hours** — after each session, and by appointment via Moodle

Your week-2 checklist

```
Before week 3
├── Finish the Lab 2 tables & report
├── Submit the report — 2nd class after lab
├── Submit ai-log.md (Session 02 entry)
├── Read DDIA 2e ch. 6–7 (9–10 before week 3)
├── Read the Raft paper: Figure 2 and §5
└── Form your project team of 2–3

Optional, if curious
├ play an interactive Raft demo online
├ read Brewer's CAP retrospective
└ ask the AI to explain PACELC — then grade it
```

Next week — Storage for Scale: HDFS and object storage, Parquet & ORC (columnar) vs Avro (row-oriented), and the NoSQL family — where this week's principles become products.

Key Takeaways

"Distribution is a set of trade-offs, not a set of features — a good engineer knows the price of everything."

✓ What we saw today

- Partitioning spreads the data — the key decides everything
- Replication buys safety, pays in lag or conflicts
- Consistency is a contract — chosen per operation
- Consensus is quorums plus rules — Raft makes them legible
- CAP covers partitions; PACELC adds everyday latency

✓ Your action plan

- Finish the Lab 2 report and the AI benchmark verdict
- Submit report + ai-log.md by the 2nd class after the lab
- Read DDIA 2e ch. 6–7, then 9–10; Raft Fig. 2 and §5
- Form your project team of 2–3 by week 3
- Next week: storage for scale — HDFS, Parquet & NoSQL

Thank you — see you in lab 2!